

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«ТЮМЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
ШКОЛА КОМПЬЮТЕРНЫХ НАУК

ОТЧЕТ
О РЕЗУЛЬТАТАХ ОЗНАКОМИТЕЛЬНОЙ ПРАКТИКИ

ТЮМГУ
(наименование организации)

АКАДЕМИЧЕСКИЙ ДЕПАРТАМЕНТ (ШКН)
(наименование структурного подразделения)

Выполнил
обучающийся 2 курса,
ОП-АТ-ПИ-24.01 группы

Авдеева Анфиса Андреевна

(подпись)

(ФИО)

Руководитель практики
от школы

Кабардинский Евгений
Олегович

(подпись)

(ФИО)

Руководитель практики
от профильной организации

(подпись)

(ФИО)

Тюмень, 2026

Защищен _____ 20__

Результаты
экзамена / зачета _____

(подпись руководителя практики от школы)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1. Инкапсуляция. Пример	Error! Bookmark not defined.
1.1 Практика: Сбои	Error! Bookmark not defined.
2. Наследование и полиморфизм. Пример	6
2.1 Практика: НоММ	6
2.2 Практика: Геометрия-2	7
3. Generics	9
3.1 Практика: Роботы	9
4. Делегирование	11
4.1 Практика: Генератор отчетов	11
5. Рефлексия	13
5.1 Практика: Дифференцирование	13
6. DDD	14
6.1 Практика: TaxiOrder	14
7. FluentAPI	16
7.1 Практика: GraphViz	16
ЗАКЛЮЧЕНИЕ	18
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	19

ВВЕДЕНИЕ

Базой учебной практики был выбран Академический департамент (ШКН). В рамках учебного плана 2 курса 09.03.03 Прикладная информатика: разработка информационных систем бизнеса была пройдена практика в период с 09.06.2026 по 24.06.2026.

Целью прохождения практики являлась изучение основ проектирования на C# путем выполнения практических заданий в среде Ulearn, а также освоение навыков визуализации архитектуры разрабатываемых решений с помощью текстовой нотации Mermaid для построения UML-диаграмм.

Для достижения поставленной цели потребовалось решение следующих задач:

- изучить теоретический материал по ключевым темам курса проектирования на C# в среде Ulearn, а именно: наследование и полиморфизм, Generics, делегирование, рефлексия, а также основы предметно-ориентированного проектирования (DDD) и построения FluentAPI;
- проанализировать практические примеры по каждой из указанных тем, разобрать предложенные в Ulearn кейсы для выявления сценариев применения каждого механизма;
- разработать собственные программные решения на C# для практических заданий курса, применяя изученные концепции на практике, а также построить UML-диаграммы для визуализации структуры классов, иерархий наследования и взаимодействия объектов.

Отчет проверил доцент Академического департамента (ШКН) Кабардинский Евгений Олегович.

Инкапсуляция. Пример

Практика: Сбои

1. Описание предметной области и сущностей

Система предназначена для анализа критических сбоев оборудования. Необходимо определить, какие устройства имели серьёзные сбои до определённой даты.

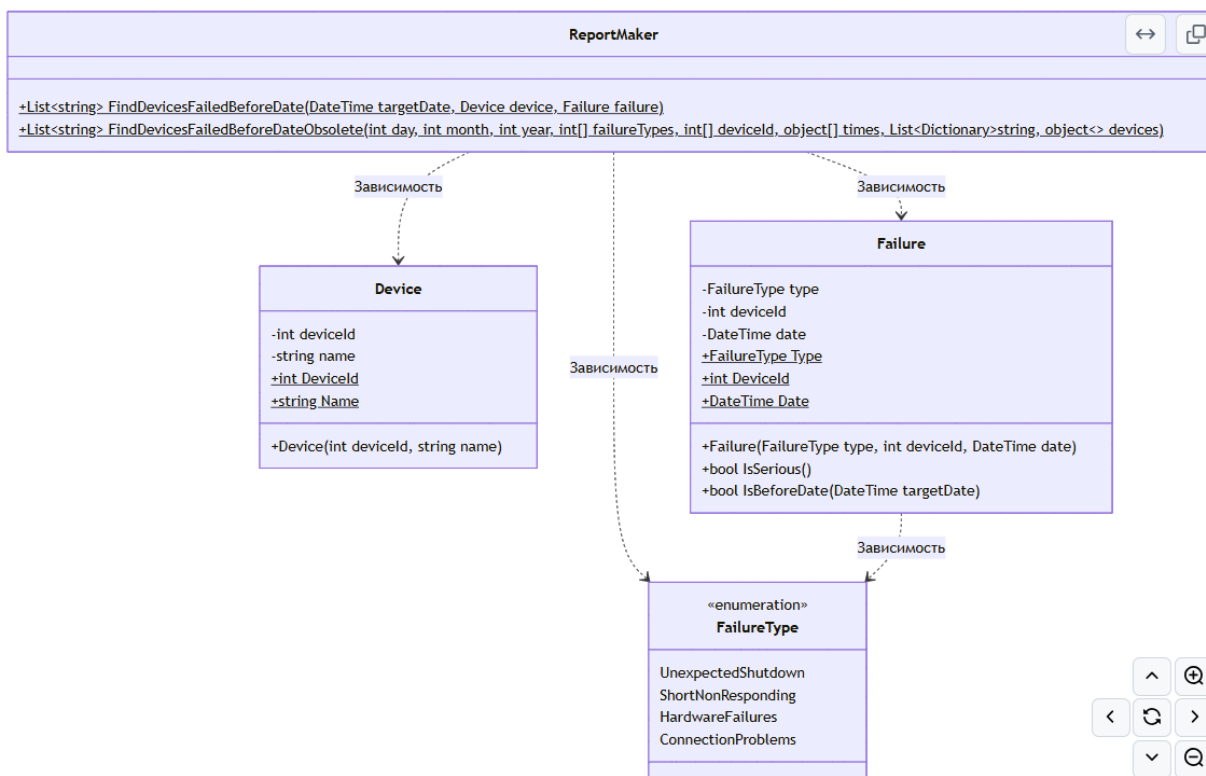
Device – устройство, хранит идентификатор DeviceId и название Name.

Failure – содержит тип сбоя FailureType, идентификатор устройства DeviceId и дату Date. Предоставляет методы IsSerious() и IsBeforeDate().

FailureType – типы сбоев UnexpectedShutdown (0), ShortNonResponding (1), HardwareFailures (2), ConnectionProblems (3).

ReportMaker – статический метод FindDevicesFailedBeforeDate принимает дату, одно устройство Device и один сбой Failure.

2. Диаграмма классов (Mermaid). Сбои. Диаграмма 1



Наследование и полиморфизм. Пример

Практика: НоММ

1. Описание предметной области и сущностей

Персонаж взаимодействует с объектами карты, которые могут иметь владельца, армию и сокровища. Метод `Interaction.Make` обрабатывает любые объекты через интерфейсы `Iownable`, `IhaveArmy` и `IhaveTreasure`, проверяет возможность победы над армией, присваивает владельца и передает сокровища игроку.

`Player` – игрок. Он может побеждать армии, получать сокровища, присваивать объекты и погибать при поражении.

`Interaction` – обработка взаимодействия игрока с объектами карты.

`Iownable` – интерфейс объектов, могут принадлежать игроку и иметь владельца.

`IhaveArmy` – интерфейс объектов, содержат армию и могут участвовать в бою с игроком.

`IhaveTreasure` – интерфейс объектов, содержат сокровища и могут быть получены игроком.

`Dwelling` – объект карты, может принадлежать игроку.

`Mine` – объект карты, может принадлежать игроку, содержит армию и сокровища.

`Creeps` – объект карты, содержит армию и сокровища, не имеет владельца.

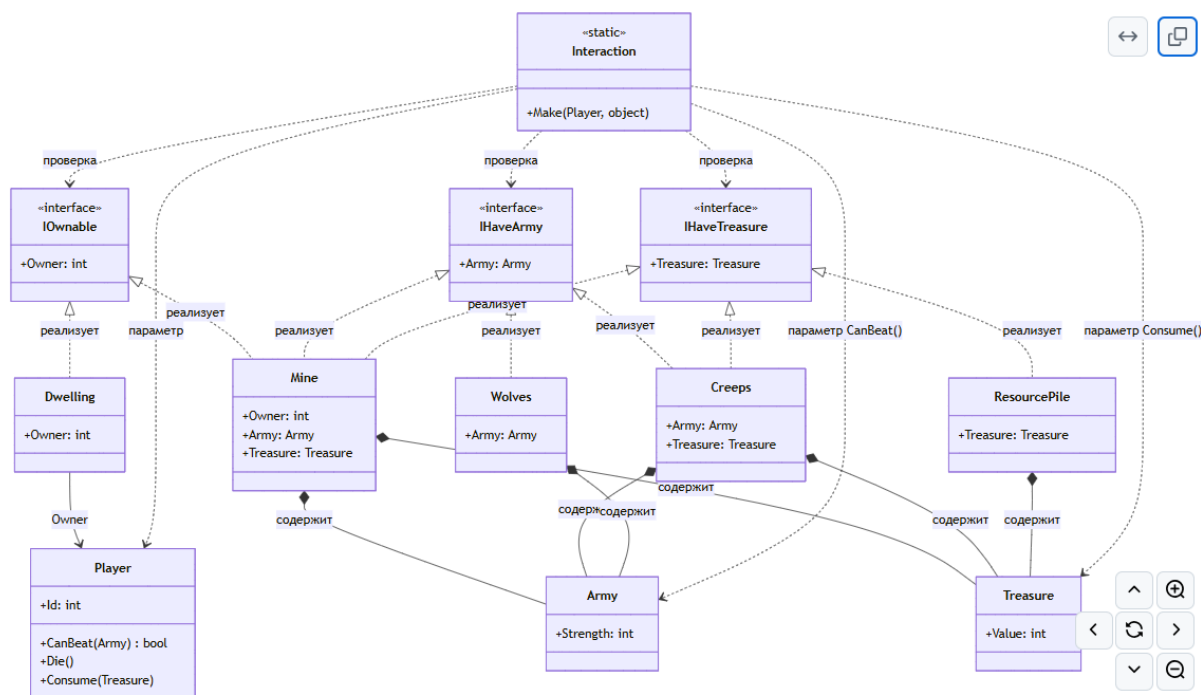
`Wolves` – объект карты, содержит только армию.

`ResourcePile` – объект карты, содержит только сокровища.

`Army` – сущность, боевая сила объекта.

`Treasure` – сущность, описывает описывающая награду игрока.

2. Диаграмма классов (Mermaid). НоММ. Диаграмма 2



Практика: Геометрия-2

1. Описание предметной области и сущностей

Рассматриваются геометрические тела в 3D: шар, параллелепипед, цилиндр и составное тело из коллекции частей. Над ними выполняются операции через классы-посетители, что позволяет добавлять новые операции без изменения классов фигур.

Body – абстрактный базовый класс, содержит позицию фигуры в пространстве и абстрактный метод **Ассерт** для принятия посетителя.

Ball – шар, наследник **Body**, имеет радиус.

RectangularCuboid – прямоугольный параллелепипед, наследник **Body**, имеет размеры по трём осям **SizeX**, **SizeY**, **SizeZ**.

Cylinder – цилиндр, наследник **Body**, имеет радиус и высоту **SizeZ**.

CompoundBody – составное тело, наследник **Body**, содержит список других тел (**Parts**).

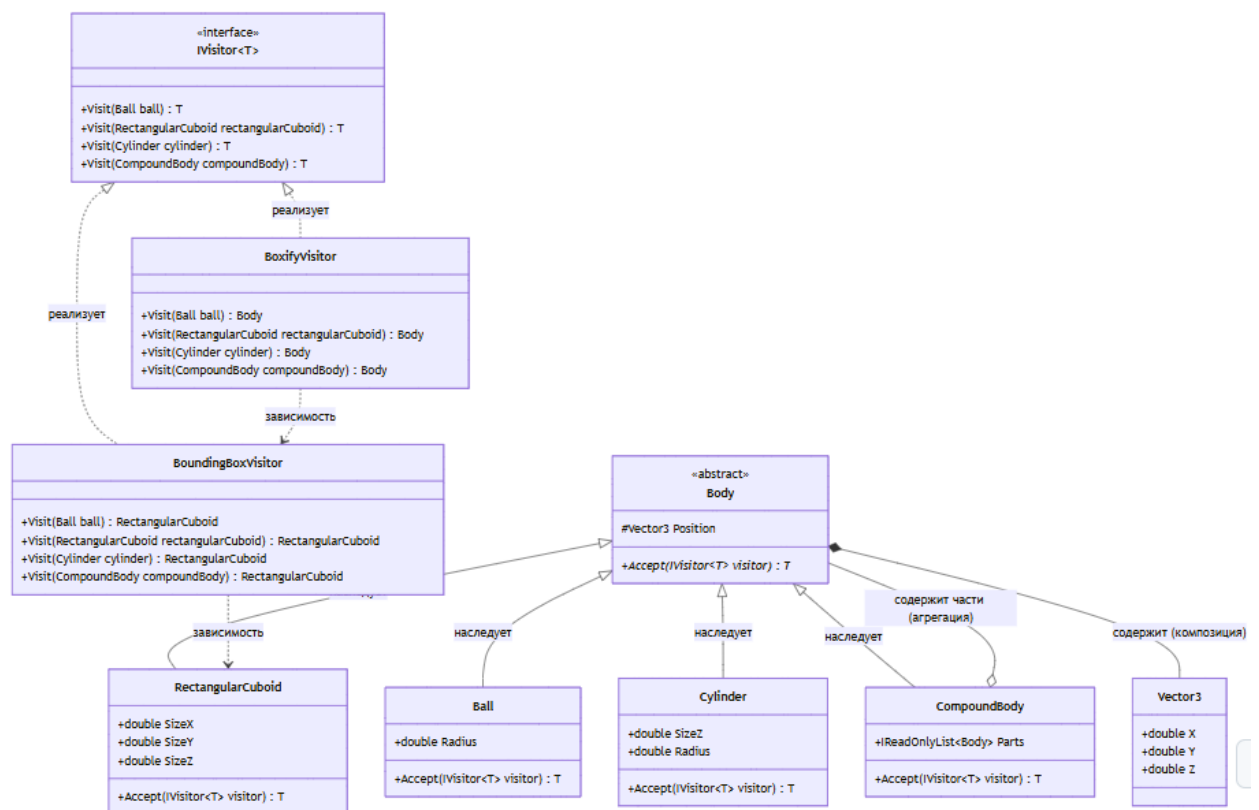
Vector3 – вспомогательная структура, хранящая координаты **X**, **Y**, **Z**.

Ivisitor<T> - интерфейс, определяющий методы Visit для каждого типа фигуры.

BoundingBoxVisitor – посетитель, реализует Ivisitor<RectangularCuboid>, вычисляет минимальный ограничивающий параллелепипед для фигуры.

BoxifyVisitor – посетитель, реализует Ivisitor<Body>, заменяет каждое простое тело на его ограничивающий параллелепипед.

2. Диаграмма классов (Mermaid). Геометрия-2. Диаграмма 3



Generics

Практика: Роботы

1. Описание предметной области и сущностей

Система управления роботами, состоит из двух компонентов: интеллектуальной системы управления (AI) и исполнительного устройства (Device). AI генерирует команды, а Device их выполняет.

IrobotAI – интерфейс искусственного интеллекта робота. Определяет метод получения команды для выполнения.

RobotAI – абстрактный базовый класс искусственного интеллекта. Содержит общий контракт для генерации команд и служит основой для реализаций AI.

ShooterAI – реализация искусственного интеллекта боевого робота. Генерирует команды типа IshooterMoveCommand.

BuilderAI – реализация искусственного интеллекта строительного робота. Генерирует команды типа ImoveCommand.

Idevice – интерфейс устройства робота. Определяет метод выполнения полученной команды.

Device – абстрактный базовый класс устройства. Задаёт общий контракт для исполнения команд различных типов.

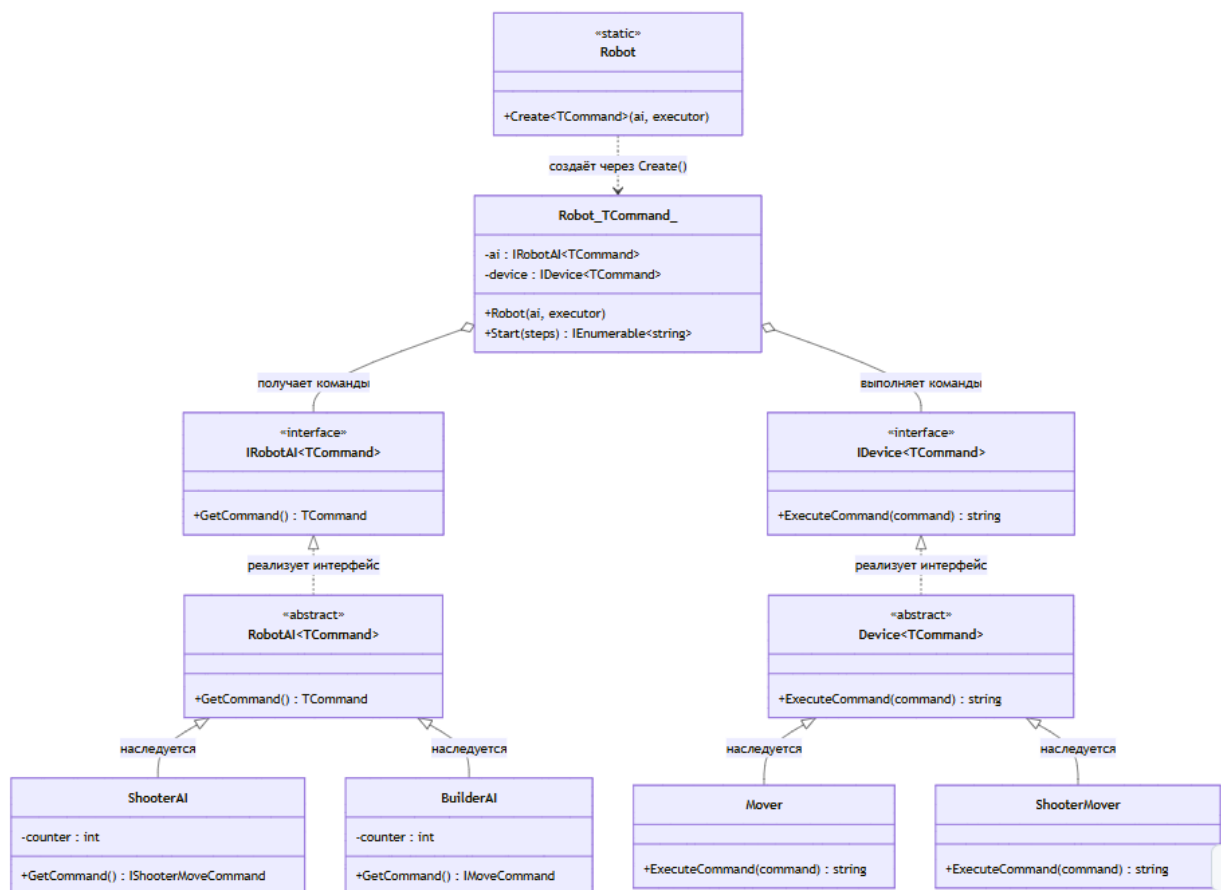
Mover – устройство перемещения робота. Выполняет команды типа ImoveCommand.

ShooterMover – устройство перемещения боевого робота. Выполняет команды типа IshooterMoveCommand.

Robot – основной класс робота. Объединяет искусственный интеллект и устройство, организуя процесс получения и выполнения команд.

Robot – статический класс. Предоставляет метод Create, который создает экземпляры роботов.

2. Диаграмма классов (Mermaid). Роботы. Диаграмма 4



Делегирование

Практика: Генератор отчетов

1. Описание предметной области и сущностей

Система генерации статистических отчетов о погодных измерениях. Вычисляет статистические показатели (среднее, стандартное отклонение, медиану) по двум параметрам (температура и влажность) за несколько дней и выводит результаты в форматах HTML, Markdown.

IstatisticsMaker – интерфейс вычислителя статистики, определяет название и метод вычисления

MeanAndStdStatistics – вычисляет среднее и стандартное отклонение, реализует IstatisticsMaker

MedianStatistics – вычисляет медиану набора чисел, реализует IstatisticsMaker

IreportFormatter – интерфейс форматтера отчета, определяет методы оформления (заголовок, списки, элементы)

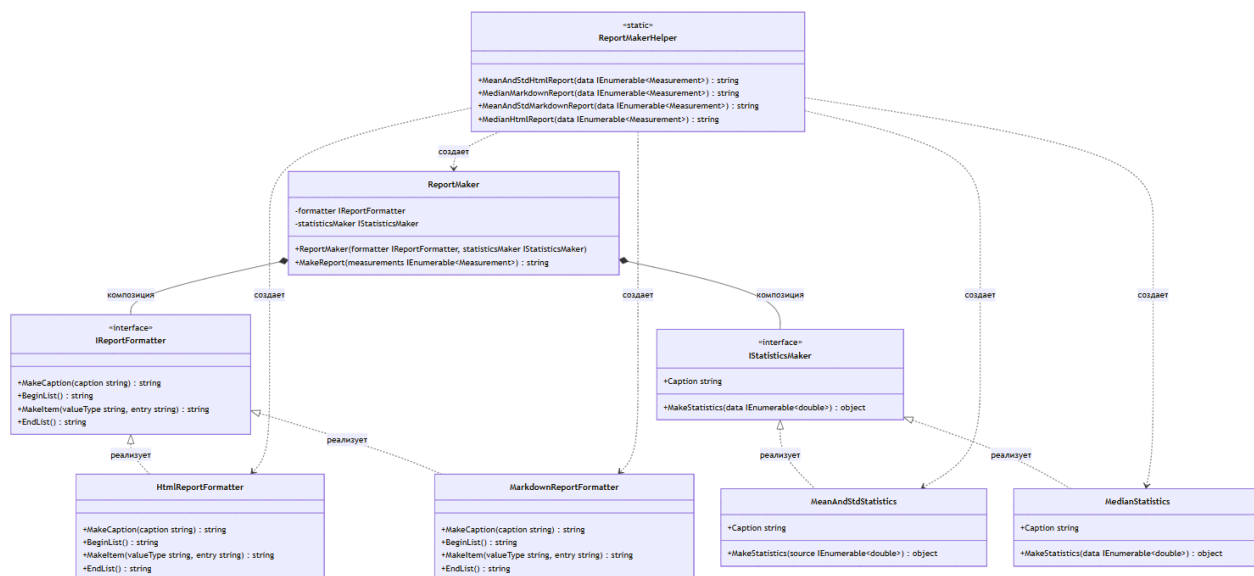
HtmlReportFormatter – форматтер для HTML, реализует IreportFormatter

MarkdownReportFormatter – форматтер для Markdown, реализует IreportFormatter

ReportMaker – главный класс, комбинирует форматтер и вычислитель для создания отчета

ReportMakerHelper – статическая фабрика с 4 готовыми методами для всех комбинаций форматов и статистик

2. Диаграмма классов (Mermaid). Генератор отчетов. Диаграмма 5



Рефлексия

Практика: Дифференцирование

1. Описание предметной области и сущностей

Программа преобразует математическое выражение в его производную, при этом обходя дерево выражения и применяя правила дифференцирования.

Algebra — главный класс, содержит логику дифференцирования. Принимает выражение, рекурсивно обходит его узлы и возвращает новое выражение (производную).

Expression — абстрактный базовый класс для всех узлов дерева выражения. Представляет собой математическую формулу.

ConstantExpression — константа (число). При дифференцировании превращается в 0.

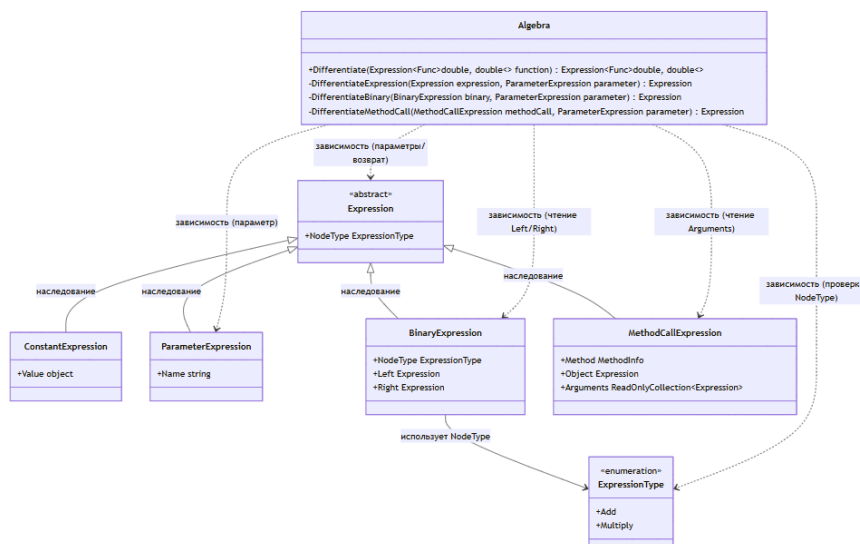
ParameterExpression — переменная, по которой дифференцируем. При дифференцировании превращается в 1.

BinaryExpression — бинарный оператор с левым и правым операндом. Поддерживаются сложение и умножение.

MethodCallExpression — вызов математической функции. Поддерживаются Sin и Cos.

ExpressionType — перечисление типов узлов. Используется для определения оператора (Add, Multiply) в бинарных выражениях.

2. Диаграмма классов (Mermaid). Дифференцирование. Диаграмма 6



DDD

Практика: TaxiOrder

1. Описание предметной области и сущностей

Система управления заказами такси. Система позволяет клиентам создавать заказы, назначать водителей, отслеживать статус выполнения заказа, управлять поездкой и завершать её. Основными участниками процесса являются клиенты, водители и диспетчерская служба.

ValueType - абстрактный базовый класс для всех объектов-значений, обеспечивает сравнение по содержимому свойств через Equals, GetHashCode, ToString

Entity - абстрактный базовый класс для всех сущностей, обеспечивает наличие Id и сравнение по нему

ITaxiApi - интерфейс, определяет контракт внешнего API для управления заказами такси

PersonName - объект-значение, хранит имя и фамилию клиента или водителя

Address - объект-значение, хранит улицу и номер дома для адреса отправления или назначения

Car - объект-значение, хранит модель, цвет и госномер автомобиля водителя

Driver - сущность водителя, содержит идентификатор, имя и автомобиль

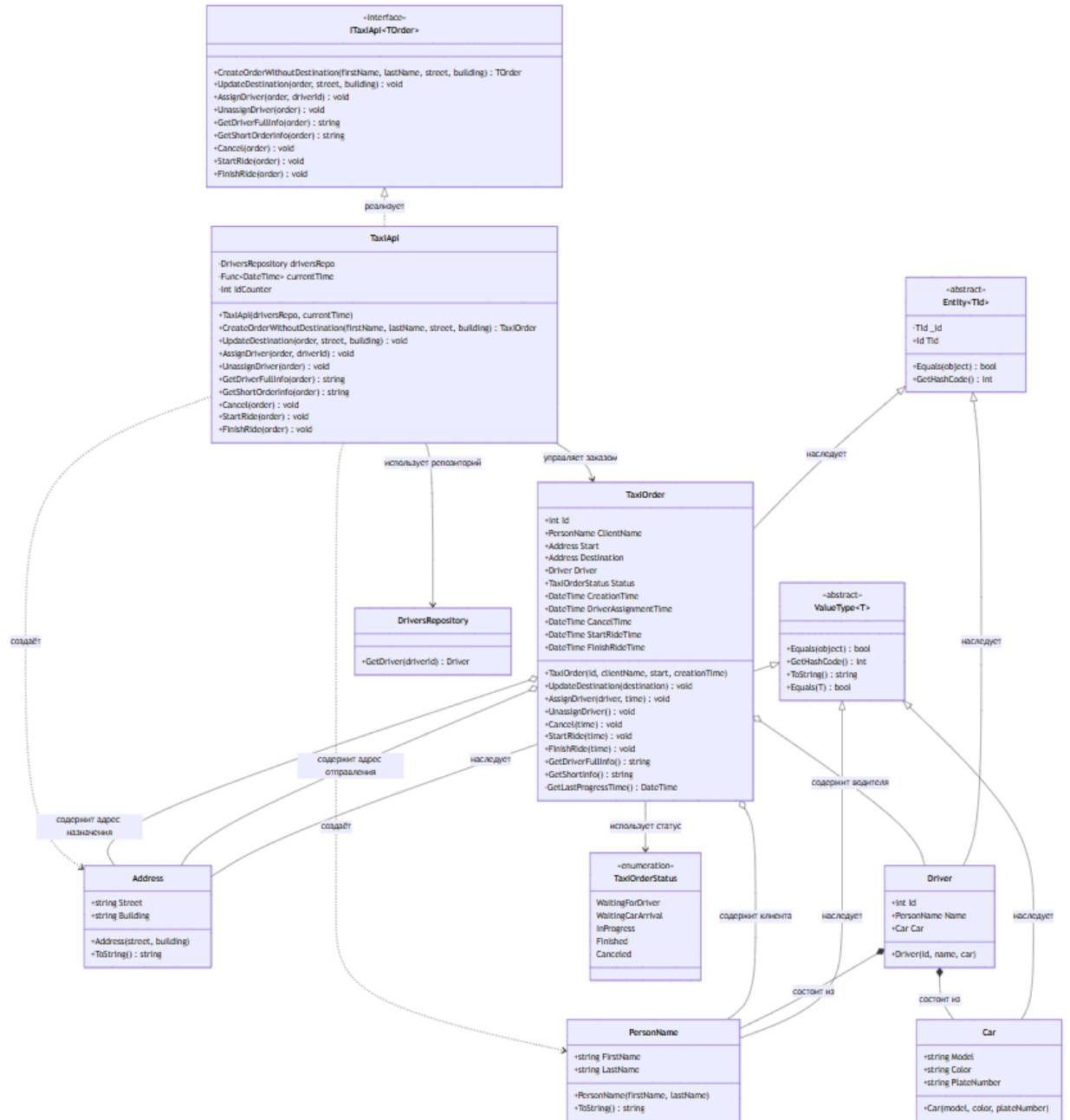
TaxiOrder - корень агрегата, управляет жизненным циклом заказа такси, хранит клиента, адреса, водителя, статус и временные метки событий

DriversRepository - репозиторий для получения данных о водителях из базы данных по идентификатору

TaxiApi - фасадный класс, предоставляет внешний интерфейс, делегирует вызовы методам TaxiOrder

TaxiOrderStatus - перечисление состояний заказа: WaitingForDriver, WaitingCarArrival, InProgress, Finished, Canceled

2. Диаграмма классов (Mermaid). TaxiOrder. Диаграмма 7



FluentAPI

Практика: GraphViz

1. Описание предметной области и сущностей

Система предназначена для построения графов в формате DOT с использованием Fluent API. Она позволяет создавать ориентированные и неориентированные графы, добавлять вершины и рёбра с атрибутами, а затем генерировать текстовое представление графа в формате, совместимом с GraphViz.

`INodeAttributes` - интерфейс настройки атрибутов вершины: цвет, размер шрифта, метка, форма. Поддерживает цепочку вызовов.

`IEdgeAttributes` - интерфейс настройки атрибутов ребра: цвет, размер шрифта, метка, вес. Поддерживает цепочку вызовов.

`INodeBuilder` - построитель вершины до применения атрибутов. Содержит метод `With` для настройки, методы добавления элементов и `Build`.

`INodeConfigured` - построитель вершины после применения атрибутов. Не содержит `With`, предотвращает повторную настройку.

`IEdgeBuilder` - построитель ребра до применения атрибутов. Содержит метод `With` для настройки, методы добавления элементов и `Build`.

`IEdgeConfigured` - построитель ребра после применения атрибутов. Не содержит `With`, предотвращает повторную настройку.

`DotGraphBuilder` - точка входа в библиотеку. Статические методы `DirectedGraph/UndirectedGraph` создают граф. Методы `AddNode/AddEdge` добавляют элементы, `Build` генерирует DOT-представление.

`NodeBuilder` - внутренняя реализация `INodeBuilder`. Хранит `Graph` и `GraphNode`. `With` создаёт `NodeAttributes` и возвращает `NodeConfigured`.

`NodeConfigured` - внутренняя реализация `INodeConfigured`. Хранит `Graph`. Не имеет `With`. Создаёт новые билдеры через `AddNode/AddEdge`.

`EdgeBuilder` - внутренняя реализация `IEdgeBuilder`. Хранит `Graph` и `GraphEdge`. `With` создаёт `EdgeAttributes` и возвращает `EdgeConfigured`.

EdgeConfigured - внутренняя реализация IEdgeConfigured. Хранит Graph. Не имеет With. Создаёт новые билдеры через AddNode/AddEdge.

NodeAttributes - внутренняя реализация INodeAttributes. Хранит GraphNode. Записывает атрибуты в словарь вершины. Возвращает this для цепочки вызовов.

EdgeAttributes - внутренняя реализация IEdgeAttributes. Хранит GraphEdge. Записывает атрибуты в словарь ребра. Возвращает this для цепочки вызовов.

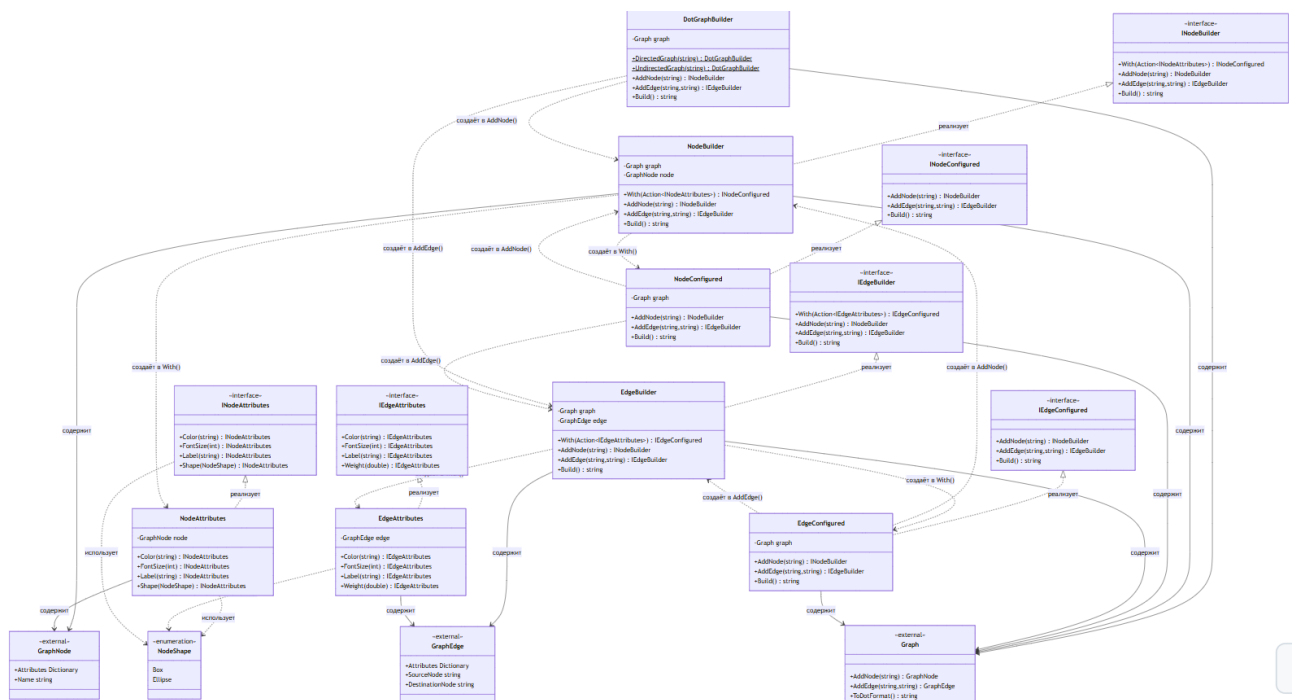
Graph - внешний класс-хранилище графа. Содержит коллекции вершин и рёбер. Методы AddNode/AddEdge для добавления, ToDotFormat для генерации DOT.

GraphNode - внешний класс вершины. Содержит словарь атрибутов и свойство Name.

GraphEdge - внешний класс ребра. Содержит словарь атрибутов, свойства SourceNode, DestinationNode и флаг Directed.

NodeShape - перечисление форм вершины: Box и Ellipse. Используется в INodeAttributes.

2. Диаграмма классов (Mermaid). GraphViz. Диаграмма 8



ЗАКЛЮЧЕНИЕ

В ходе прохождения ознакомительной практики были выполнены все поставленные цели и задачи.

В рамках практики изучены теоретические основы ключевых тем курса «Проектирование на C#»: наследование и полиморфизм, Generics, делегирование, рефлексия, DDD и FluentAPI. Полученные знания были закреплены на практике путем выполнения заданий в среде Ulearn по каждой из указанных тем. Для разработанных решения были построены UML-диаграммы классов, что позволило наглядно документировать архитектуру спроектированных систем.

В результате практики сформированы практические навыки объектно-ориентированного проектирования на языке C#, визуального моделирования программных систем.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Спецификация Mermaid : диаграммы классов // Mermaid : [сайт]. – URL: <https://mermaid.js.org/syntax/classDiagram.html> (дата обращения: 09.06.2026).
2. Курс «Проектирование на C#» // Ulearn : [сайт]. – URL: https://ulearn.me/Course/cs2/Vvedenie_7df81bac-f52c-4219-8663-4d215bafbc7a (дата обращения: 09.06.2026).
3. Посетитель (шаблон проектирования) // Википедия : свободная энциклопедия. – URL: [https://ru.wikipedia.org/wiki/Посетитель_\(шаблон_проектирования\)](https://ru.wikipedia.org/wiki/Посетитель_(шаблон_проектирования)) (дата обращения: 10.06.2026).
4. Двоичное дерево поиска // Википедия : свободная энциклопедия. – URL: https://ru.wikipedia.org/wiki/Двоичное_дерево_поиска (дата обращения: 10.06.2026).
5. Observer pattern // Wikipedia : the free encyclopedia. – URL: https://en.wikipedia.org/wiki/Observer_pattern (дата обращения: 11.06.2026).
6. Graphviz : [сайт]. – URL: <https://www.graphviz.org/> (дата обращения: 12.06.2026).
7. DOT Language // Graphviz : [сайт]. – 28.09.2024. – URL: <https://graphviz.gitlab.io/doc/info/lang.html> (дата обращения: 12.06.2026).
8. Gephi : [сайт]. – URL: <https://gephi.org/> (дата обращения: 13.06.2026).